

Gestión de Proyectos de Inteligencia Artificial

Actividad 8: Evidencia de implementación



Universidad
Tecmilenio®

Gibran Martínez Figueroa

19/07/2026

Actividad 8:

Evidencia de implementación

Objetivo: Implementar la monitorización, mantenimiento y gobernanza operativa, asegurando la continuidad y confiabilidad del servicio. Al finalizar, deberás proporcionar evidencia documentada de cada fase del proceso, incluyendo el uso adecuado de herramientas y la simulación de incidentes críticos.

Introducción

El monitoreo de modelos de Machine Learning es una actividad importante cuando estos se encuentran en producción, ya que permite supervisar su desempeño y detectar posibles problemas que puedan afectar su funcionamiento. Contar con un monitoreo adecuado ayuda a garantizar la disponibilidad, confiabilidad y calidad de las predicciones generadas por el modelo.

En esta evidencia de implementación se utilizará un modelo de clasificación de aprobación de créditos desarrollado con Random Forest, simulando un escenario donde una institución financiera evalúa solicitudes de crédito de sus clientes. Este ejemplo permite observar el comportamiento de un modelo desplegado y aplicar las prácticas de monitoreo propuestas en la actividad.

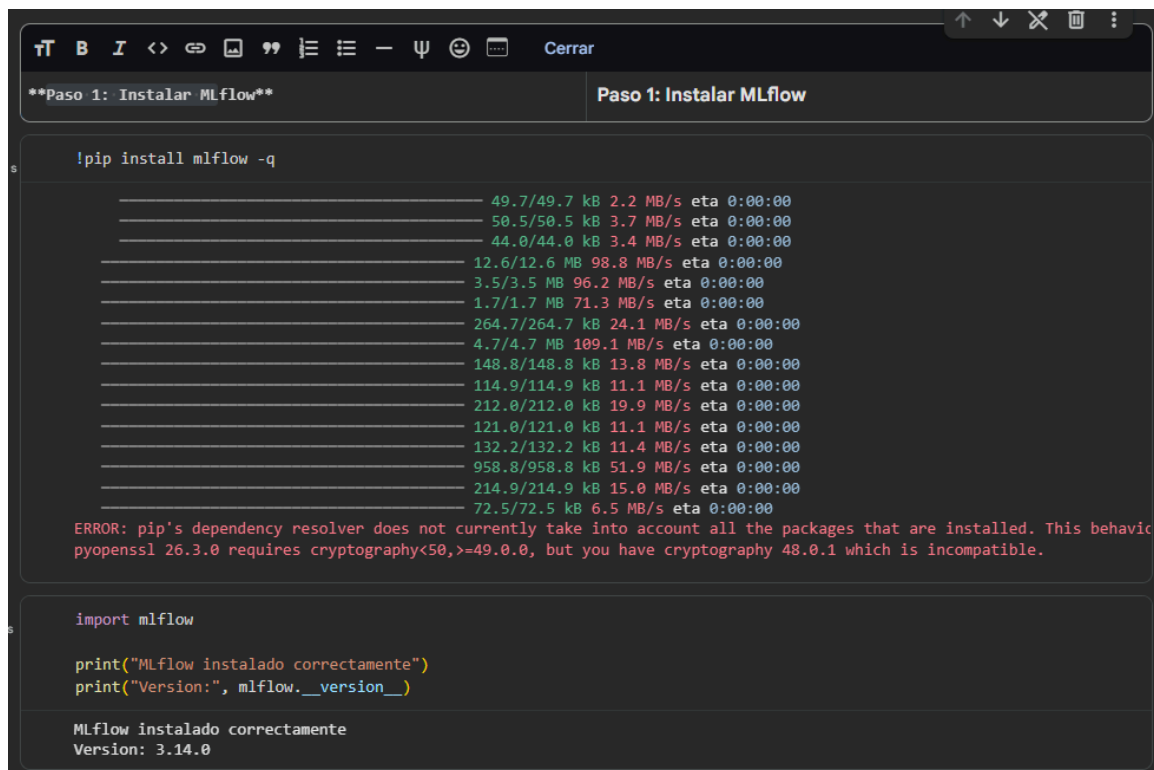
Como herramienta principal se utilizará MLflow, la cual permite registrar y visualizar métricas, comparar ejecuciones, almacenar información de los modelos y dar seguimiento a su rendimiento. Gracias a estas funcionalidades es posible supervisar el funcionamiento del modelo y analizar su comportamiento a lo largo del tiempo.

A través de las evidencias presentadas en este documento se mostrarán las métricas registradas, los dashboards generados y las alertas configuradas para monitorear el modelo. Además, se realizarán simulaciones de incidentes con el objetivo de validar que los mecanismos de monitoreo y respuesta funcionan correctamente ante posibles fallas o degradaciones del servicio. Esto permitirá demostrar la aplicación de prácticas modernas de observabilidad y gobernanza operativa en sistemas basados en inteligencia artificial, garantizando la continuidad y calidad del servicio ofrecido.

Evidencia de implementación

Instalación de librerías y MLflow

En este paso se instalaron las dependencias requeridas para el desarrollo de la actividad. Aunque se presentó una advertencia de compatibilidad entre algunas librerías, la instalación se completó correctamente y no afectó la ejecución del modelo ni el registro de métricas en MLflow.



```
!pip install mlflow -q

49.7/49.7 kB 2.2 MB/s eta 0:00:00
50.5/50.5 kB 3.7 MB/s eta 0:00:00
44.0/44.0 kB 3.4 MB/s eta 0:00:00
12.6/12.6 MB 98.8 MB/s eta 0:00:00
3.5/3.5 MB 96.2 MB/s eta 0:00:00
1.7/1.7 MB 71.3 MB/s eta 0:00:00
264.7/264.7 kB 24.1 MB/s eta 0:00:00
4.7/4.7 MB 109.1 MB/s eta 0:00:00
148.8/148.8 kB 13.8 MB/s eta 0:00:00
114.9/114.9 kB 11.1 MB/s eta 0:00:00
212.0/212.0 kB 19.9 MB/s eta 0:00:00
121.0/121.0 kB 11.1 MB/s eta 0:00:00
132.2/132.2 kB 11.4 MB/s eta 0:00:00
958.8/958.8 kB 51.9 MB/s eta 0:00:00
214.9/214.9 kB 15.0 MB/s eta 0:00:00
72.5/72.5 kB 6.5 MB/s eta 0:00:00

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behavior is deprecated. pyopenssl 26.3.0 requires cryptography<50,>=49.0.0, but you have cryptography 48.0.1 which is incompatible.

import mlflow

print("MLflow instalado correctamente")
print("Version:", mlflow.__version__)

MLflow instalado correctamente
Version: 3.14.0
```

Generar datos simulados

Se generó un conjunto de datos sintético con 5,000 registros y 10 variables de entrada. Estos datos representan información utilizada para evaluar solicitudes de crédito y fueron utilizados para entrenar y validar el modelo de clasificación.

Paso 3: Generar datos simulados

```
X, y = make_classification(  
    n_samples=5000,  
    n_features=10,  
    n_informative=6,  
    n_redundant=2,  
    random_state=42  
)  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)  
  
print("Datos cargados correctamente")
```

... Datos cargados correctamente

Para visualizar los datos simulados cree el siguiente código, Los valores generados son simulados y fueron creados únicamente para fines de entrenamiento y validación del modelo.

```
import pandas as pd  
  
df_credits = pd.DataFrame(X, columns=[  
    "Edad",  
    "Ingreso_Mensual",  
    "Antiguedad_Laboral",  
    "Score_Crediticio",  
    "Deuda_Actual",  
    "Numero_Creditos",  
    "Historial_Pagos",  
    "Monto_Solicitado",  
    "Dependientes",  
    "Ahorros"  
)  
  
df_credits["Aprobacion"] = y  
  
display(df_credits.head(10))
```

Score_Crediticio	Deuda_Actual	Numero_Creditos	Historial_Pagos	Monto_Solicitado	Dependientes	Ahorros	Aprobacion
-0.215829	-1.419279	0.914109	-5.980685	1.518092	-0.042540	-3.397662	0
-0.091221	-1.360185	-0.606968	-4.648657	-2.454777	-0.667688	-0.629240	0
1.562327	4.191341	1.230190	3.491643	0.827346	0.635312	-2.593035	0
-2.495924	-1.738007	-1.256467	-1.842087	1.668228	0.494795	-0.994083	0
-2.204839	3.360553	0.922288	0.683308	-1.889515	-1.772399	1.312052	1
0.132640	1.556043	2.848289	-0.275154	1.231785	-0.073907	-1.435493	0
0.907064	2.106804	-0.562708	-1.721881	2.511537	-0.276346	-3.193861	0
-0.371494	2.595255	0.375995	5.083844	0.657958	-0.407438	2.588078	1
0.764342	-3.846012	-0.012996	-2.741570	-1.145605	-0.012714	-1.379041	0
-2.558883	0.609122	-0.837658	-4.086636	-3.214651	0.469214	0.146754	1

Esta validación permite confirmar la cantidad de registros y variables disponibles para el entrenamiento del modelo, asegurando que la información se haya generado correctamente antes de iniciar el proceso de aprendizaje.

Y un código para ver la forma del dataset:

```
▶ print("Cantidad de registros:", df.shape[0])  
  print("Cantidad de variables:", df.shape[1] - 1)  
  
  print("\nDistribución de aprobaciones:")  
  print(df["Aprobacion"].value_counts())
```

*** Cantidad de registros: 5000
Cantidad de variables: 10

Distribución de aprobaciones:

Aprobacion	
1	2503
0	2497

Name: count, dtype: int64

Entrenamiento del modelo

```
with mlflow.start_run():

    model = RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_metric("precision", precision)
    mlflow.log_metric("recall", recall)
    mlflow.log_metric("f1_score", f1)

    mlflow.sklearn.log_model(
        model,
        "random_forest_model"
    )

    print("Accuracy:", round(accuracy,4))
    print("Precision:", round(precision,4))
    print("Recall:", round(recall,4))
    print("F1 Score:", round(f1,4))

... 2026/07/20 03:34:09 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/07/20 03:34:09 INFO mlflow.store.db.utils: Updating database tables
2026/07/20 03:34:14 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
Accuracy: 0.921
Precision: 0.9264
Recall: 0.9133
F1 Score: 0.9198
```

Después del entrenamiento se obtuvieron las métricas principales del modelo. Los resultados muestran un Accuracy de 92.10%, Precision de 92.64%, Recall de 91.33% y F1 Score de 91.98%, indicando un desempeño estable y consistente para el escenario simulado.

Simulación de alertas y data drift

Se realizaron pruebas para simular condiciones que podrían afectar el rendimiento del modelo. Cuando los valores configurados superan los umbrales establecidos, el sistema genera alertas que permiten identificar oportunamente posibles problemas en producción.

Incidente 1: Degradación del Modelo

Durante las pruebas se simuló una degradación del modelo estableciendo un valor de Accuracy por debajo del umbral definido del 85%.

Incidente 2: Data Drift

Se simuló un valor PSI de 0.28, superior al límite permitido de 0.25, lo que indica cambios importantes entre los datos de entrenamiento y los datos recibidos en producción.

▼ Paso 5: Simulación de alertas

[6]
✓ 0 s

```
accuracy_actual = accuracy

if accuracy_actual < 0.85:
    print("ALERTA: Accuracy por debajo del umbral")
else:
    print("Modelo funcionando correctamente")
```

▼

Modelo funcionando correctamente

▼ Paso 6: Simulación de Data Drift

[7]
✓ 0 s

```
psi = 0.28

if psi > 0.25:
    print("ALERTA: Data Drift detectado")
else:
    print("Sin Data Drift")
```

▼

ALERTA: Data Drift detectado

Dashboard

El dashboard permite visualizar de manera gráfica las métricas principales obtenidas durante la evaluación del modelo. Esta información facilita el monitoreo continuo y ayuda a detectar cambios en el desempeño del servicio.

Paso 7: Crear dashboard

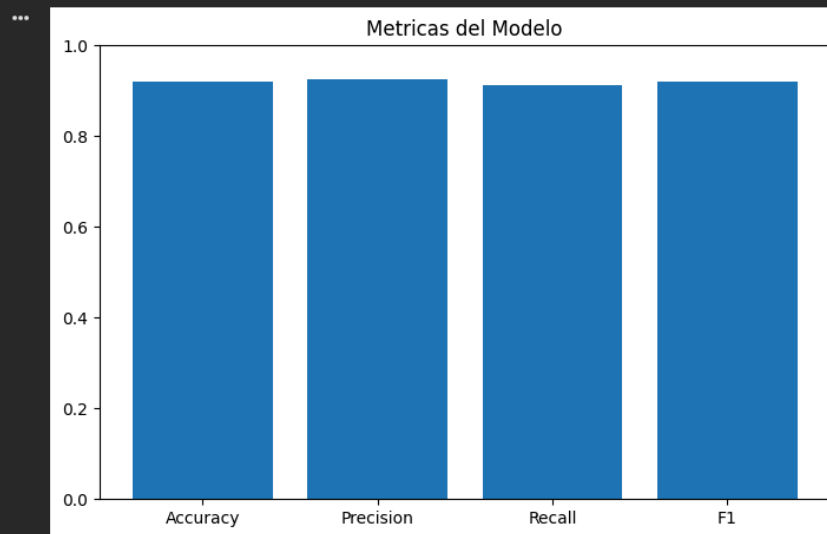
```
import matplotlib.pyplot as plt

metricas = [
    accuracy,
    precision,
    recall,
    f1
]

nombres = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1"
]

plt.figure(figsize=(8,5))
plt.bar(
    nombres,
    metricas
)

plt.ylim(0,1)
plt.title("Metricas del Modelo")
plt.show()
```



Conclusión

Durante esta actividad se implementó una estrategia básica de monitoreo para un modelo de clasificación desarrollado con Random Forest, utilizando MLflow como herramienta principal para registrar y visualizar métricas. A partir de los datos simulados fue posible entrenar el modelo y evaluar su desempeño mediante indicadores como Accuracy, Precision, Recall y F1 Score.

Los resultados obtenidos muestran un Accuracy de 92.10%, lo que indica que el modelo es capaz de realizar predicciones correctas en la mayoría de los casos evaluados. De igual forma, las métricas de Precision, Recall y F1 Score se mantuvieron por encima del 91%, demostrando un comportamiento estable dentro del escenario simulado.

Además, se realizaron pruebas relacionadas con alertas y detección de Data Drift, permitiendo comprobar que es posible identificar situaciones que podrían afectar el rendimiento del modelo durante su operación. Finalmente, el dashboard generado facilitó la visualización de las métricas más importantes y permitió tener una visión general del estado del modelo.

Los incidentes simulados permitieron validar que las alertas configuradas son capaces de identificar problemas que pueden afectar el rendimiento del modelo. Asimismo, los runbooks definidos proporcionan una guía para responder de forma estructurada ante estos eventos, facilitando la recuperación del servicio y manteniendo la calidad de las predicciones.

En conclusión, la implementación realizada demuestra cómo las prácticas de monitoreo y observabilidad ayudan a supervisar el comportamiento de un modelo de Machine Learning en producción, facilitando la detección temprana de problemas y contribuyendo a mantener la calidad y confiabilidad del servicio.